
Fechete Serban Tudor
30431

**Motorcycle Service Web App
Vision**

Version 1.0

Motorcycle Service Web App	Version: 1.0
Vision	Date: 17/05/2026

Revision History

Date	Version	Description	Author
17/05/2026	1.0	Initial draft for Motorcycle Service Web App	Fechete Serban Tudor

Motorcycle Service Web App	Version: 1.0
Vision	Date: 17/05/2026

Table of Contents

1.	Introduction	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms, and Abbreviations	4
1.4	References	4
1.5	Overview	4
2.	Positioning	4
2.1	Problem Statement	4
2.2	Product Position Statement	4
3.	Stakeholder and User Descriptions	5
3.1	Stakeholder Summary	5
3.2	User Summary	5
3.3	User Environment	6
4.	Product Requirements	6

Motorcycle Service Web App	Version: 1.0
Vision	Date: 17/05/2026

Vision

1. Introduction

1.1 Purpose

The purpose of this document is to collect, analyze, and define the high-level needs and core features of the Motorcycle Service Web App. It focuses on the specific capabilities required by garage administrators and motorcycle owners, explaining the operational and business justifications behind these needs.

1.2 Scope

This Vision document applies directly to the development of the multi-tier management system designed for a motorcycle service center or garage. The system encompasses a web-based client application, a secure REST API backend, and a relational persistence layer, affecting day-to-day workshop coordination, customer tracking, and automated digital invoicing.

1.3 Definitions, Acronyms, and Abbreviations

SPA (Single Page Application): A web application that loads a single HTML page and dynamically updates it as the user interacts with the app, built here via Angular.

REST API: Representational State Transfer Application Programming Interface, built using Spring Boot to expose backend endpoints to the client layer.

JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties, used here for stateless authentication.

ORM (Object-Relational Mapping): A technique used to query and manipulate data from a database using an object-oriented paradigm, implemented using Hibernate.

DTO (Data Transfer Object): An object that carries data between processes to hide core database entities.

RBAC (Role-Based Access Control): A method of restricting system access to authorized users based on specific roles (e.g., ADMIN, CUSTOMER).

1.4 References

Motorcycle Service Web App Analysis and Design Document: Assignment 3, Fechete Serban Tudor, Group 30431, UTCN, 2025.

Project Supplementary Specification Document, Version 1.0, 2026.

1.5 Overview

This document is structured to provide an immediate summary of the application's market positioning and product definitions. Section 2 breaks down the core problem statement and product alignment. Section 3 Profiles the stakeholders and end-user personas, while Section 4 outlines the overarching technological platform and performance baselines required by the software architecture.

2. Positioning

2.1 Problem Statement

The problem of	<i>Manual, unoptimized management of service schedules, customer profiles, inventory parts usage, and invoice calculations.</i>
affects	<i>Garage administrators, mechanics, and customer motorcycle owners.</i>
the impact of which is	<i>Operational scheduling bottlenecks, poor history tracking across multiple vehicles, administrative data entry errors, and</i>

Motorcycle Service Web App	Version: 1.0
Vision	Date: 17/05/2026

	<i>a lack of transparency for clients tracking their repair status.</i>
a successful solution would be	<i>A unified, secure, layered web app that streamlines client registration , tracks multi-bike ownership , automates appointment scheduling , links part allocation, and offers digital PDF invoice generation.</i>

2.2 Product Position Statement

For	<i>Motorcycle service centers, workshop mechanics, and their registered clientele.</i>
Who	<i>Require a secure, reliable ecosystem to handle repair intakes, vehicle service histories, and automated, transparent billing.</i>
The Motorcycle Service Web App	<i>Is a containerized, full-stack service management system.</i>
That	<i>Enforces secure data workflows through strict DTO validation , tracks historical mechanic actions, and enables isolated customer downloads of cryptographically compiled service invoices.</i>
Unlike	<i>Generic, non-relational scheduling spreadsheets or manual paper billing methods.</i>
Our product	<i>Features a dedicated domain model optimized for vehicle properties, state-of-the-art JWT security, database persistence via Docker volumes, and server-side PDF compilation preventing client-side price tampering.</i>

3. Stakeholder and User Descriptions

3.1 Stakeholder Summary

<i>Workshop Director</i>	<i>The business owner is funding and managing the service facility.</i>	<i>Defines core business success metrics, approves platform deployment budgets, and ensures the digital invoice workflows match regional tax standards.</i>
<i>IT Security Auditor</i>	<i>External or internal system overseer focused on compliance.</i>	<i>Mandates that the application protects client information, enforces encryption of data at rest/in transit, and avoids OWASP vulnerabilities.</i>

Motorcycle Service Web App	Version: 1.0
Vision	Date: 17/05/2026

3.2 User Summary

<i>ADMIN</i>	<i>Garage staff, receptionists, or active shop mechanics.</i>	<i>Executes full CRUD operations on customer profiles , tracks vehicle metrics , updates part allocations , alters appointment lifecycles, and updates repair statuses.</i>	<i>Workshop Director</i>
<i>CUSTOMER</i>	<i>Motorcycle owners seeking service updates.</i>	<i>Accesses the specialized client dashboard , views their specific registered motorcycles, checks ongoing appointment statuses, and securely downloads completed invoices.</i>	<i>N/A</i>

3.3 User Environment

Admin Interfaces: Primarily operated on fixed desktop terminals or ruggedized tablets located in noisy, high-activity mechanical bays or front-desk reception areas. Handled across multiple active browser tabs.

Customer Interfaces: Accessed on personal computing environments, including home desktops, tablets, and mobile smartphone browsers, demanding highly responsive web design.

Integration Ecosystems: The presentation layer communicates seamlessly across network boundaries via automated HTTP/HTTPS requests directed to a backend REST API backed by a secure containerized network.

4. Product Requirements

Frontend Engine: Implemented using Angular to construct a clean, modern Single Page Application client.

Backend Application Server: Structured via Java and the Spring Boot framework using an automated, multi-tiered architecture (Controllers, Services, Repositories).

Persistence Layer: Uses a dedicated PostgreSQL relational instance to guarantee relational integrity across all database tables (part, person, motorcycle, service appointment).

Data Integration & Validation: Managed via Hibernate ORM with integrated database-level constraints, including non-nullable properties, unique email verification parameters, and strict backend DTO data mapping.

Deployment Constraints: The entire multi-tiered platform must compile, bind, and operate within an integrated multi-container Docker environment utilizing host-mapped volumes for persistent storage.